

RealMLP Implementation — Block-by-Block Explanation

This document explains each logical block of the RealMLP training cell in `cleaner.ipynb` (the PyTorch MLP added to the notebook). Explanations are concise and tied to the code structure as found in the RealMLP cell.

Header & Imports:

- Purpose: bring in libraries used by the model and training loop.
- Key items:
 - `torch, torch.nn, Dataset, DataLoader`: PyTorch model, layers and data utilities.
 - `StandardScaler`: scale numeric features.
 - `StratifiedKFold, roc_auc_score`: cross-validation and evaluation.
 - `numpy, pandas, gc`: array/table handling and garbage collection.

Device selection:

- `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')` selects GPU when available, otherwise CPU.
- Ensures tensors and model are moved to the same device for training and inference.

Prepare dataset copies:

- `X_nn = X[FEATURES].copy(), test_nn = test[FEATURES].copy(), y_nn = y.copy()`
- Purpose: avoid mutating global `X`, `test`, `y` used elsewhere; work on local copies.

Categorical factorization (NO target encoding):

- Iterates over `CATS` and factorizes categories using a combined train+test series so train/test share consistent integer codes.
- Stores cardinalities per category in `cat_cardinalities` for embedding sizes.
- Avoids any target-derived statistics (fulfills the "no target encoding" requirement).

Numeric features cast & missing handling:

- Converts numeric columns in `NUMS` to `float32` and fills missing values with the training column mean.
- Keeps test fill values consistent by using the train mean.

Scaling numeric features:

- `StandardScaler` is fit on train numeric features and applied to test numeric features.
- Produces `X_nn_nums` and `test_nn_nums` arrays for model input (or empty arrays if no numeric features).

Prepare categorical numpy arrays:

- Converts factorized categorical columns to integer NumPy arrays (`x_nn_cats`, `test_nn_cats`).
- These arrays are later used for embedding lookups in the model.

Dataset class (`TabularDataset`):

- Lightweight PyTorch Dataset that returns (`nums`, `cats`, `label`) for training/validation and (`nums`, `cats`) for test (when `labels=None`).
- Ensures numeric arrays are float32 and categorical arrays are int64 for PyTorch compatibility.

Model class (`RealMLP`):

- Structure:
 - Builds an `nn.ModuleList` of `nn.Embedding` layers, one per categorical feature, with per-feature embedding dimension `ed` computed as `min(emb_dim, max(2, (card+1)//2))`.
 - `cat_output_dim` accumulates the total embedding dimension across all categorical features.
 - Concatenates embeddings with numeric input to form the MLP input.
 - MLP is a sequence of linear -> batchnorm -> ReLU -> dropout blocks per `hidden_dims`, finishing with a linear layer to a single logit output.
- Forward pass:
 - Look up embeddings for each categorical feature and concatenate them.
 - Concatenate numeric features and embeddings, feed through `self.mlp`, return 1-D logits vector.

Training loop setup (folds, arrays, hyperparams):

- `n_splits`, `StratifiedKFold` ensure stratified cross-validation.
- `oof_preds_nn` and `test_preds_nn` initialize arrays for out-of-fold and averaged test predictions.
- Hyperparameters are set: `batch_size`, `epochs`, `patience`, optimizer LR and weight decay inside the cell.

Per-fold data split & DataLoaders:

- For each fold, numeric and categorical arrays are indexed to create train/validation splits.
- `TabularDataset` instances and `DataLoaders` are created for train, val, and test. `num_workers=0` keeps it safe across environments.

Model initialization, optimizer, loss:

- Instantiates `RealMLP` with the numeric feature count and `cat_cardinalities`.
- Uses Adam optimizer with specified learning rate and weight decay.
- Uses `BCEWithLogitsLoss` because the model outputs logits for a binary classification task.

Epoch / training step:

- Standard training loop: set `model.train()`, iterate batches, move tensors to device, compute logits, compute loss, backpropagate, and optimizer step.
- Numeric (`xb_num`) tensors are float, categorical (`xb_cat`) tensors are long indices for embeddings, and labels `yb` are float.

Validation & early stopping:

- After each epoch run, model is evaluated on the validation loader with `model.eval()` and `torch.no_grad()`.
- Collects logits for validation, converts to probabilities with the sigmoid function, computes AUC via `roc_auc_score`.
- Keeps track of the best validation AUC, saves `model.state_dict()` for the best epoch to `best_realmpl_fold.pth`.
- Implements simple patience-based early stopping using `best_epoch` and `patience`.

Loading best model & fold predictions:

- Loads the saved best model for the fold, computes validation predictions and writes them into `oof_preds_nn[val_idx]`.
- Computes test predictions for the fold and accumulates into `test_preds_nn` (averaged over folds).

Cleanup & aggregation:

- Deletes model and loaders at end of fold and calls `gc.collect()` to free memory.
- After all folds, computes `roc_auc_score` on the full OOF predictions for a final OOF AUC report.

Save submission and OOF files:

- Writes `submission_realmpl.csv` containing averaged test probabilities.
 - Writes `oof_realmpl.csv` with `id`, `true` target and `pred` columns for stacking/analysis.
-